

Лабораторна робота №4.

Розробка програми генератора паролів

Завдання:

1. Використовуючи приклад напишіть скрипт генератор випадкових паролів. Розберіться як працює даний скрипт.
2. Збережіть скрипт під назвою *password_generator.py* та запустіть даний скрипт у терміналі: *python3 password_generator.py --help*. Переконайтеся, що скрипт працює коректно.
3. Використовуючи даний скрипт:
 - згенеруйте 2 паролі довжиною 10 символів;
 - згенеруйте 3 цифрові паролі довжиною 8 цифр;
 - згенеруйте пароль з 7 символів нижнього регістру, 3 великих, 5 цифр і 2 спеціальних символів;
 - запишіть всі згенеровані паролі у *pass.txt*.

Приклад

```
1  from argparse import ArgumentParser
2  import secrets
3  import random
4  import string
5
6  # Setting up the Argument Parser
7  parser = ArgumentParser(
8      prog='Password Generator.',
9      description='Generate any number of passwords with this tool.'
10 )
11
12 # Adding the arguments to the parser
13 parser.add_argument("-n", "--numbers", default=0, help="Number of digits in the PW", type=int)
14 parser.add_argument("-l", "--lowercase", default=0, help="Number of lowercase chars in the PW", type=int)
15 parser.add_argument("-u", "--uppercase", default=0, help="Number of uppercase chars in the PW", type=int)
16 parser.add_argument("-s", "--special-chars", default=0, help="Number of special chars in the PW", type=int)
17
18 # add total pw length argument
19 parser.add_argument("-t", "--total-length", type=int,
20                     help="The total password length. If passed, it will ignore -n, -l, -u and -s, " \
21                     "and generate completely random passwords with the specified length")
22
23
24 # The amount is a number so we check it to be of type int.
25 parser.add_argument("-a", "--amount", default=1, type=int)
26 parser.add_argument("-o", "--output-file")
27
```

```

28 # Parsing the command line arguments.
29 args = parser.parse_args()
30
31 # list of passwords
32 passwords = []
33
34 # Looping through the amount of passwords.
35 for _ in range(args.amount):
36     if args.total_length:
37         # generate random password with the length
38         # of total length based on all available characters
39         passwords.append("".join(
40             [secrets.choice(string.digits + string.ascii_letters + string.punctuation) \
41              for _ in range(args.total_length)]))
42     else:
43         password = []
44
45         # If / how many numbers the password should contain
46         for _ in range(args.numbers):
47             password.append(secrets.choice(string.digits))
48
49         # If / how many uppercase characters the password should contain
50         for _ in range(args.uppercase):
51             password.append(secrets.choice(string.ascii_uppercase))
52

```

```

53         # If / how many lowercase characters the password should contain
54         for _ in range(args.lowercase):
55             password.append(secrets.choice(string.ascii_lowercase))
56
57         # If / how many special characters the password should contain
58         for _ in range(args.special_chars):
59             password.append(secrets.choice(string.punctuation))
60
61         # Shuffle the list with all the possible letters, numbers and symbols.
62         random.shuffle(password)
63
64         # Get the letters of the string up to the length argument and then join them.
65         password = ''.join(password)
66
67         # append this password to the overall list of password.
68         passwords.append(password)
69
70 # Store the password to a .txt file.
71 if args.output_file:
72     with open(args.output_file, 'w') as f:
73         f.write('\n'.join(passwords))
74
75 print('\n'.join(passwords))

```